

操作系统课程设计

说明书

学 院： 信息科学与工程学院

一、 理解 Nachos 模拟的物理机的运行机制.....	3
1. Sysdep 模块分析（文件 sysdep.cc sysdep.h）	5
2. 中断模块分析（文件 interrupt.cc interrupt.h）	8
3. 时钟中断模块分析（文件 timer.cc timer.h）	12
4. 终端设备模块分析（文件 console.cc console.h）	错误！未定义书签。
二、 理解 Nachos 中线程运行机制.....	14
1. 工具模块分析（文件 list.cc list.h utility.cc utility.h）	17
2. 线程启动和调度模块分析（文件 switch.s switch.h）	18
3. 线程模块分析（文件 thread.cc thread.h）	19
4. 线程调度算法模块分析（文件 scheduler.cc scheduler.h）	22
5. Nachos 主控模块分析（文件 main.cc system.cc system.h）	23
6. 同步机制模块分析（文件 synch.cc synch.h）	24
三、 理解 Nachos 中支持用户进程的机制.....	26
一、 用户程序空间（文件 address.cc, address.h）	29
二、 系统调用（文件 exception.cc, syscall.h, start.s）	32

一、理解 Nachos 模拟的物理机的运行机制

Machine 类用来模拟计算机主机。它提供的功能有：读写寄存器。读写主存、运行一条用户程序的汇编指令、运行用户程序、单步调试用户程序、显示主存和寄存器状态、将虚拟内存地址转换为物理内存地址、陷入 Nachos 内核等等。

Machine 类实现方法是在宿主机上分配两块内存分别作为虚拟机的寄存器和物理内存。运行用户程序时，先将用户程序从 Nachos 文件系统中读出，写入模拟的物理内存中，然后调用指令模拟模块对每一条用户指令解释执行。将用户程序的读写内存要求，转变为对物理内存地址的读写。**Machine** 类提供了单步调试用户程序的功能，执行一条指令后会自动停下来，让用户查看系统状态，不过这里的单步调试是汇编指令级的，需要读者对 R2/3000 指令比较熟悉。如果用户程序想使用操作系统提供的功能或者发出异常信号时，**Machine** 调用系统异常陷入功能，进入 Nachos 的核心部分。

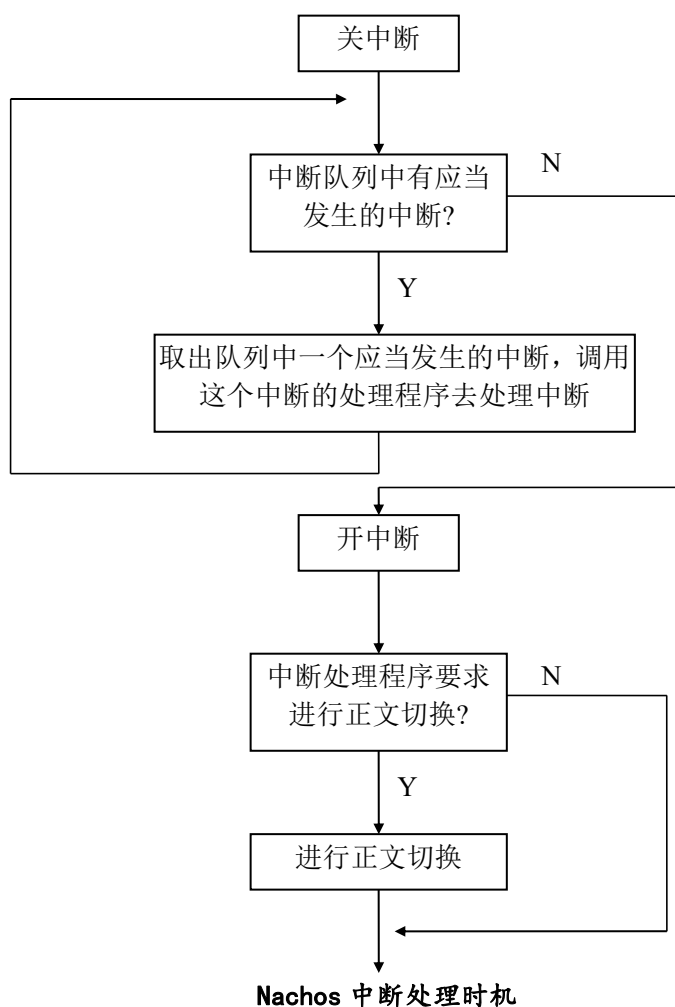
Interrupt 类用来模拟硬件中断系统。在这个中断系统中，中断状态有开，关两种，中断类型有时钟中断、磁盘中断、控制台写中断、控制台读中断、网络发送中断以及网络接收中断。机器状态有用户态，核心态和空闲态。中断系统提供的功能有开/关中断，读/写机器状态，将一个即将发生中断放入中断队列，以及使机器时钟前进一步。

在 **Interrupt** 类中有一个记录即将发生中断的队列，称为中断等待队列。中断等待队列中每个等待处理的中断包含中断类型、中断处理程序的地址及参数、中断应当发生的时间等信息。一般是由硬件设备模拟程序把将要发生的中断放入中断队列。中断系统提供了一个模拟机器时钟，机器时钟在下列情况下前进：

- 用户程序打开中断
- 执行一条用户指令
- 处理机没有进程正在运行

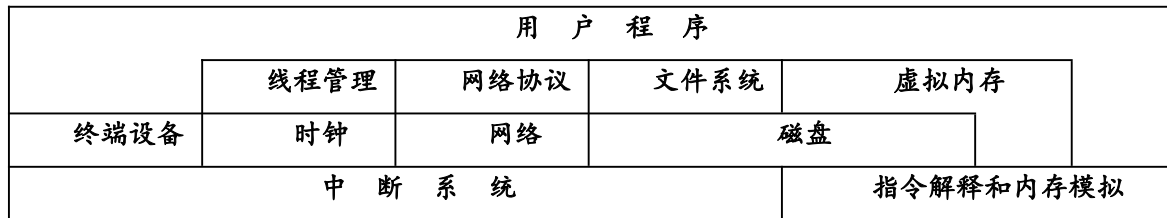
机器时钟前进时，中断处理的过程如下图所示：

本设计有全套完整的资料：包括源程序、数据库，说明书，答辩PPT等,联系QQ：1415736481 获取！
也可代做其它题目的毕业设计



中断系统成为整个 Nachos 虚拟机的基础，其它的模拟硬件设备都是建立在中断系统之上的。在此之上，加上 Machine 类模拟的指令解释器，可以实现 Nachos

的线程管理、文件系统管理、虚拟内存、用户程序和网络管理等所有操作系统功能。下图展示了 Nachos 系统的整体结构。



Nachos 系统的整体结构

机器模拟的实现：

1. Sysdep模块分析（文件sysdep.cc sysdep.h）

Nachos 的运行环境可以是多种操作系统，由于每种操作系统所提供的系统调用或函数调用在形式和内容上可能有细微的差别。sysdep 模块的作用是屏蔽掉这些差别。

1.1 PoolFile 函数

语法：bool PoolFile (int fd)
参数：fd:文件描述符，也可以是一个套接字 (socket)
功能：测试一个打开文件 fd 是否有内容可以读，如果有则返回 TRUE，否则返回 FALSE。
实现：当 Nachos 系统处于 IDLE 状态时，测试过程有一个延时，也就是在一定时间范围内如果有内容可读的话，同样返回 TRUE。
实现：通过 select 系统调用。
返回：打开文件是否有内容供读取。

1.2 OpenForWrite 函数

语法：int OpenForWrite (char *name)
参数：name: 文件名
功能：为写操作打开一个文件。如果该文件不存在，产生该文件；如果该文件已经存在，则将该文件原有的内容删除。
实现：通过 open 系统调用。
返回：打开的文件描述符。

1.3 OpenForReadWrite 函数

语法: `int OpenForReadWrite (char *name, bool crashOnError)`

参数: `name`: 文件名
`crashOnError`: crash 标志

功能: 为读写操作打开一个文件。当 `crashOnError` 标志设置而文件不能读写打开时，系统出错退出。

实现: 通过 `open` 系统调用。

返回: 打开的文件描述符。

1.4 Read 函数

语法: `void Read (int fd, char *buffer, int nBytes)`

参数: `fd`: 打开文件描述符
`buffer`: 读取内容的缓冲区
`nBytes`: 需要读取的字节数

功能: 从一个打开文件 `fd` 中读取 `nBytes` 的内容到 `buffer` 缓冲区。如果读取失败，系统退出。

实现: 通过 `read` 系统调用。

返回: 空。

1.5 ReadPartial 函数

语法: `int ReadPartial (int fd, char *buffer, int nBytes)`

参数: `fd`: 打开文件描述符
`buffer`: 读取内容的缓冲区
`nBytes`: 需要读取的最大字节数

功能: 从一个打开文件 `fd` 中读取 `nBytes` 的内容到 `buffer` 缓冲区。

实现: 通过 `read` 系统调用。

返回: 实际读出的字节数。

1.6 WriteFile 函数

语法: `void WriteFile (int fd, char *buffer, int nBytes)`

参数: `fd`: 打开文件描述符
`buffer`: 需要写的内容所在的缓冲区
`nBytes`: 需要写的内容最大字节数

功能: 将 `buffer` 缓冲区中的内容写 `nBytes` 到一个打开文件 `fd` 中。

实现: 通过 `write` 系统调用。

返回: 空

1.7 Lseek 函数

语法: `void Lseek (int fd, int offset, int whence)`

参数: `fd`: 文件描述符
`offset`: 偏移量
`whence`: 指针移动的起始点

功能: 移动一个打开文件的读写指针，含义同 `lseek` 系统调用；出错则退出系统。

实现: 通过 `lseek` 系统调用。

返回： 空。

1.8 Tell 函数

语法： int Tell (int fd)
参数： fd: 文件描述符
功能： 指出当前读写指针位置
实现： 通过 lseek 系统调用。
返回： 返回当前指针位置。

1.9 Close 函数

语法： void Close (int fd)
参数： fd: 文件描述符
功能： 关闭当前打开文件 fd，如果出错则退出系统。
实现： 通过 close 系统调用。
返回： 空

1.10 Unlink 函数

语法： bool Unlink (char *name)
参数： name: 文件名
功能： 删除文件。
实现： 通过 unlink 系统调用。
返回： 删除成功，返回 TRUE；否则返回 FALSE。

1.11 OpenSocket 函数

语法： int OpenSocket ()
参数： 无
功能： 申请一个 socket。
通过 socket 系统调用。
其中 AF_UNIX 参数说明使用 UNIX 内部协议。（Nachos 是用 SOCKET 文件来模拟网络节点，所以采用 UNIX 内部协议。向该文件读写内容分别代表从该节点读取内容和向该网络节点发送内容）
实现： SOCK_DGRAM 参数说明采用无连接定长数据包型的数据链路。
返回： 申请到的 socket ID。

1.12 CloseSocket 函数

语法： void CloseSocket (int sockID)
参数： sockID: socket 标识
功能： 释放一个 socket。
实现： 通过 close 系统调用。
返回： 无。

1.13 Abort 函数

语法： void Abort ()
参数： 无
功能： 退出系统（非正常退出）。
实现： 通过 abort 系统调用。

返回： 无。

1.14 Exit 函数

语法： void Exit (int exitCode)
参数： exitCode: 向系统的返回值
功能： 退出系统。
实现： 通过 exit 系统调用。
返回： 空

2. 中断模块分析（文件interrupt.cc interrupt.h）

中断模块的主要作用是模拟底层的中断机制。可以通过该模拟机制来启动和禁止中断 (SetLevel)；该中断机制模拟了 Nachos 系统需要处理的所有的中断，包括时钟中断、磁盘中断、终端读/终端写中断以及网络接收/网络发送中断。

中断的发生总是有一定的时间。比如当向硬盘发出读请求，硬盘处理请求完毕后会发生中断；在请求和处理完毕之间需要经过一定的时间。所以在该模块中，模拟了时钟的前进。为了实现简单和便于统计各种活动所占用的时间起见，Nachos 规定系统时间在以下三种情况下前进：

执行用户态指令，时钟前进是显而易见的。我们认为，Nachos 执行每条指令所需时间是固定的，为一个时钟单位(Tick)。

一般系统态在进行中断处理程序时，需要关中断。但是中断处理程序本身也需要消耗时间，而在关闭中断到重新打开中断之间无法非常准确地计算时间，所以当中断重新打开的时候，加上一个中断处理所需时间的平均值。

当系统中没有就绪进程时，系统处于 Idle 状态。这种状态可能是系统中所有的进程都在等待各自的某种操作完成。也就是说，系统将在未来某个时间发生中断，到中断发生的时候中断处理程序将进行中断处理。在系统模拟中，有一个中断等待队列，专门存放将来发生的中断。在这种情况下，可以将系统时间直接跳

到中断等待队列第一项所对应的时间，以免不必要的等待。

当前面两种情况需要时钟前进时，调用 **OneTick** 方法。**OneTick** 方法将系统态和用户态的时间分开进行处理，这是因为用户态的时间计算是根据用户指令为单位的；而在系统态，没有办法进行指令的计算，所以将系统态的一次中断调用或其它需要进行时间计算的单位设置为一个固定值，假设为一条用户指令执行时间的 10 倍。

虽然 Nachos 模拟了中断的发生，但是毕竟不能与实际硬件一样，中断发生的时机可以是任意的。比如当系统中没有就绪进程时，时钟直接跳到未处理中断队列的第一项的时间。这实际情况下，系统处于 **Idle** 状态到中断等待队列第一项发生时间之间，完全有可能有其它中断发生。由于中断发生的时机不是完全随机的，所以在 Nachos 系统中运行的程序，不正确的同步程序也可能正常运行，我们在此需要密切注意。

Nachos 线程运行有三种状态：

Idle 状态

系统 CPU 处于空闲状态，没有就绪线程可以运行。如果中断等待队列中有需要处理的除了时钟中断以外的中断，说明系统还没有结束，将时钟调整到发生中断的时间，进行中断处理；否则认为系统结束所有的工作，退出。

系统态

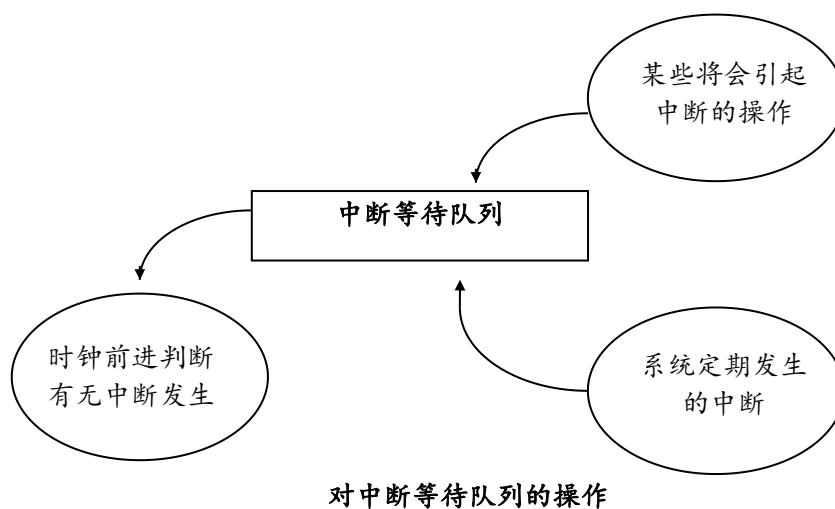
Nachos 执行系统程序。Nachos 虽然模拟了虚拟机的内存，但是 Nachos 系统程序本身的运行不是在该模拟内存中，而是利用宿主机的存储资源。这是 Nachos 操作系统同真正操作系统的重要区别。

用户态

系统执行用户程序。当执行用户程序时，每条指令占用空间是 Nachos 的模拟内存。

中断等待队列是 Nachos 虚拟机最重要的数据结构之一，它记录了当前虚拟机可以预测的将在未来发生的所有中断。当系统进行了某种操作可能引起未来发生的中断时，如磁盘的写入、向网络写入数据等都会将中断插入到中断等待队列中；对于一些定期需要发生的中断，如时钟中断、终端读取中断等，系统会在中断处理后将下一次要发生的中断插入到中断等待队列中。中断的插入过程是一个优先队列的插入过程，其优先级是中断发生的时间，也就是说，先发生的中断将优先得到处理。

当时钟前进或者系统处于 Idle 状态时，Nachos 会判断中断等待队列中是否有



本设计有全套完整的资料：包括源程序、数据库，
说明书，答辩PPT等,联系QQ：1415736481 获取！
也可代做其它题目的毕业设计

要发生的中断，如果有中断需要发生，则将该中断从中断等待队列中删除，调用相应的中断处理程序进行处理。中断处理程序是在某种特定的中断发生时被调用，中断处理程序的作用包括可以在现有的模拟硬件的基础上建立更高层次的抽象。比如现有

的模拟网络是有丢失帧的不安全网络，在中断处理程序中可以加入请求重发机制来实现一个安全网络。

2.1 PendingInterrupt类

```
class PendingInterrupt {
    public:
        PendingInterrupt (VoidFunctionPtr func, int param, int time,
IntType kind);
        VoidFunctionPtr handler;           // 中断对应的中断处理程序
        int arg;                           // 中断处理程序的参数
        int when;                           // 中断发生的时机
        IntType type;                       // 中断的类型，供调试用
};
```

这个类定义了一个中断等待队列中需要处理的中断。为了方便起见，所有类的数据和成员函数都设置为 `public` 的，不需要其它的 `Get` 和 `Set` 等存取内部数据的函数。初始化函数就是为对应的参数赋值。

2.2 Interrupt类

```
class Interrupt {
    public:                                // 以下函数是 Interrupt 的对外接口
        Interrupt();                       // 初始化中断模拟
        ~Interrupt();                      // 终止中断模拟

        IntStatus SetLevel(IntStatus level);
                                                // 开关中断，并且返回之前的状态
        void Enable();                     // 开中断
        IntStatus getLevel() {return level;}
                                                // 取回当前中断的开关状态
        void Idle();                       // 当进程就绪队列为空时，执行该函数
        void Halt();                       // 退出系统，并打印状态
        void YieldOnReturn();              // 设置中断结束后要进行进程切换的标志
        MachineStatus getStatus() { return status; }
                                                // 返回系统当前的状态
        void setStatus(MachineStatus st) { status = st; }
                                                // 设置系统当前的状态
        void DumpState();                  // 调试当前中断队列状态用

        void Schedule(VoidFunctionPtr handler, int arg, int when, IntType
```

```

type);                                // 在中断等待队列中，增加一个等待中断
    void OneTick();                    // 模拟时钟前进
private:                               // 以下是内部数据和内部处理方法
    IntStatus level;                  // 中断的开关状态
    List *pending;                    // 当前系统中等待中断队列
    bool inHandler;                   // 是否正在进行中断处理标志
    bool yieldOnReturn;               // 中断处理后是否需要正文切换标志
    MachineStatus status;             // 当前虚拟机运行状态
    bool CheckIfDue(bool advanceClock);
                                        // 检查当前时刻是否有要处理的中断
    void ChangeLevel(IntStatus old, IntStatus now);
                                        // 改变当前中断的开关状态，但不前进模拟时钟
};

```

其中，Schedule 和 OneTick 两个方法虽然标明是 public 的，但是除了虚拟机模拟部分以外的其它类方法是不能调用这两个方法的。将它们设置成 public 的原因是因为虚拟机模拟的其它类方法需要直接调用这两个方法。

3. 时钟中断模块分析（文件timer.cc timer.h）

该模块的作用是模拟时钟中断。Nachos 虚拟机可以如同实际的硬件一样，每隔一定的时间会发生一次时钟中断。这是一个可选项，目前 Nachos 还没有充分发挥时钟中断的作用，只有在 Nachos 指定线程随机切换时，（Nachos -rs 参数，见线程管理部分 Nachos 主控模块分析）启动时钟中断，在每次的时钟中断处理的最后，加入了线程的切换。实际上，时钟中断在线程管理中的作用远不止这些，时钟中断还可以用作：

- 线程管理中的时间片轮转法的时钟控制，（详见线程管理系统中的实现实例中，对线程调度的改进部分）不一定每次时钟中断都会引起线程的切换，而是由该线程是否的时间片是否已经用完来决定。
- 分时系统线程优先级的计算（详见线程管理系统中的实现实例中，对线程调度的改进部分）

- 线程进入睡眠状态时的时间计算

可以通过时钟中断机制来实现 `sleep` 系统调用，在时钟中断处理程序中，每隔一定的时间对定时睡眠线程的时间进行一次评估，判断是否需要唤醒它们。

Nachos 利用其模拟的中断机制来模拟时钟中断。时钟中断间隔由 `TimerTicks` 宏决定（100 倍 `Tick` 的时间）。在系统模拟时有一个缺陷，如果系统就绪进程不止一个的话，每次时钟中断都一定会发生进程的切换（见 `system.cc` 中 `TimerInterruptHandler` 函数）。所以运行 Nachos 时，如果以同样的方式提交进程，系统的结果将是一样的。这不符合操作系统的运行不确定性的特性。所以在模拟时钟中断的时候，加入了一个随机因子，如果该因子设置的话，时钟中断发生的时机将在一定范围内是随机的。这样有些用户程序在同步方面的错误就比较容易发现。但是这样的时钟中断和真正操作系统中的时钟中断将有不同的含义。不能象真正的操作系统那样通过时钟中断来计算时间等等。是否需要随机时钟中断可以通过设置选项(`-rs`)来实现。

`Timer` 类的实现很简单，当生成出一个 `Timer` 类的实例时，就设计了一个模拟的时钟中断。这里考虑的问题是：怎样实现定期发生时钟中断？

在 `Timer` 的初始化函数中，借用 `TimerHandler` 内部函数（见第 1 行）。为什么不直接用初始化函数中的 `timerHandler` 参数作为中断处理函数呢？因为如果直接使用 `timerHandler` 作为时钟中断处理函数，第 8 行是将一个时钟中断插入等待处理中断队列，一旦中断时刻到来，立即进行中断处理，处理结束后并没有机会将下一个时钟中断插入到等待处理中断队列。`TimerHandler` 内部函数正是处理这个问题。当时钟中断时刻到来时，调用 `TimerHandler` 函数，其调用 `TimerExpired` 方法，该方法将新的时钟中断插入到等待处理中断队列中，然后再调用真正的时

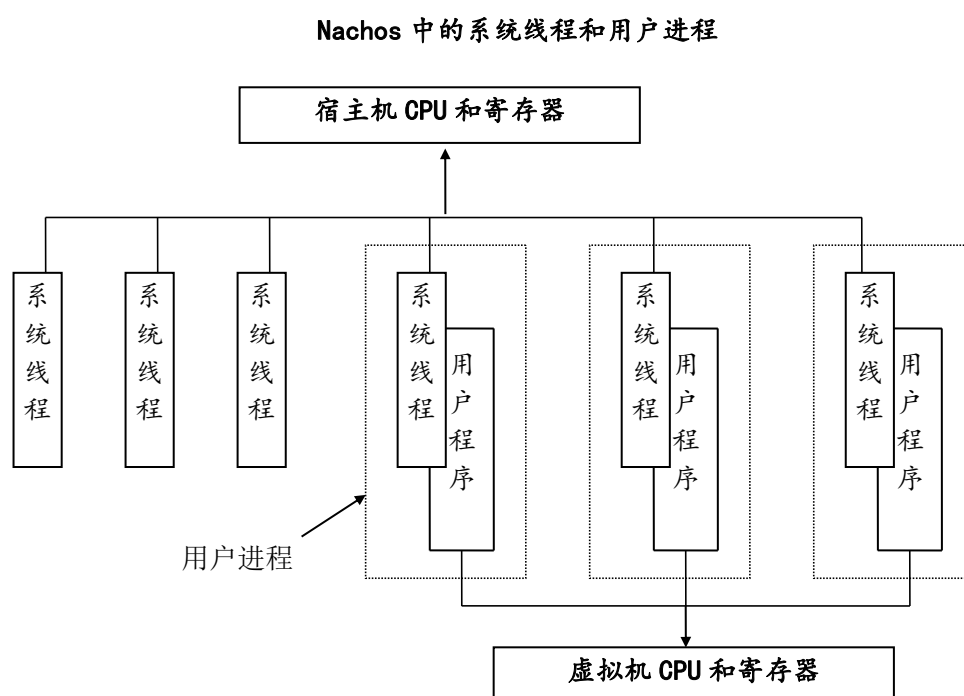
钟中断处理函数。这样 Nachos 就可以定时的收到时钟中断。

那么为什么不将 `TimerExpired` 方法作为时钟中断在 `Timer` 的初始化函数中调用呢？这是由于 C++ 语言不能直接引用一个类内部方法的指针，所以借用 `TimerHandler` 内部函数。这也是 `TimerExpired` 必须设计成 `public` 的方法的原因，因为它要被 `TimerHandler` 调用。

这样的方法不仅仅在 `Timer` 模拟时钟中断中用到，所有需要定期发生的中断都可以采用这样的方法，如 Nachos 需要定期地检查是否有终端的输入、网络是否有发给自己的报文等都是用这种方式实现。详见 `network.cc` 以及 `console.cc`。

`TimeOfextInterrupt()` 方法的作用是计算下一次时钟中断发生的时机，如果需要时钟中断发生的时机是随机的，可以在 Nachos 命令行中设置 `-rs` 选项。这样，Nachos 的线程切换的时机将会是随机的。但是此时时钟中断则不能作为系统计时的标准了。

二、理解 Nachos 中线程运行机制



Nachos 为线程提供的功能函数有:

1. 生成一个线程(Fork)
2. 使线程睡眠等待(Sleep)
3. 结束线程(Finish)
4. 设置线程状态(setStatus)
5. 放弃处理机(Yield)

线程系统的结构如图所示:



Nachos 线程系统的结构

线程管理系统中，有两个与机器相关的函数，正文切换过程依赖于具体的机器，这是因为系统线程切换是借助于宿主机的正文切换，正文切换过程中的寄存器保护，建立初始调用框架等操作对不同的处理机结构是不一样的。其中一个函数是 ThreadRoot，它是所有线程运行的入口；另一个函数是 SWITCH，它负责线程之间的切换。

Scheduler 类用于实现线程的调度。它维护一个就绪线程队列，当一个线程可以占用处理机时，就可以调用 ReadyToRun 方法把这个线程放入就绪线程队列，并把线程状态改成就绪态。FindNextToRun 方法根据调度策略，取出下一个应运

行的线程，并把这个线程从就绪线程队列中删除。如果就绪线程队列为空，则此函数返回空(NULL)。现有的调度策略是先进先出策略(FIFO)，

Thread 类的对象既用作线程的控制块，相当于进程管理中的 PCB，作用是保存线程状态、进行一些统计，又是用户调用线程系统的界面。

用户生成一个新线程的方法是：

```
                                // 生成一个线程类
->                                ,                                // 定义新线程的执行函数及其参数
```

Fork 方法分配一块固定大小的内存作为线程的堆栈，在栈顶放入 ThreadRoot 的地址。当新线程被调上 CPU 时，要用 SWITCH 函数切换线程图像，SWITCH 函数返回时，会从栈顶取出返回地址，于是将 ThreadRoot 放在栈顶，在 SWITCH 结束后就会立即执行 ThreadRoot 函数。ThreadRoot 是所有线程的入口，它会调用 Fork 的两个参数，运行用户指定的函数；Yield 方法用于本线程放弃处理机。Sleep 方法可以使当前线程转入阻塞态，并放弃 CPU，直到被另一个线程唤醒，把它放回就绪线程队列。在没有就绪线程时，就把时钟前进到一个中断发生的时刻，让中断发生并处理此中断，这是因为在没有线程占用 CPU 时，只有中断处理程序可能唤醒一个线程，并把它放入就绪线程队列。

线程要等到本线程被唤醒后，并且又被线程调度模块调上 CPU 时，才会从 Sleep 函数返回。有趣的是，新取出的就绪线程有可能就是这个要睡眠的线程。例如，如果系统中只有一个 A 线程，A 线程在读磁盘的时候会进入睡眠，等待磁盘操作完成。因为这时只有一个线程，所以 A 线程不会被调下 CPU，只是在循环语句中等待中断。当磁盘操作完成时，磁盘会发出一个磁盘读操作中断，此

中断将唤醒 A 线程，把它放入就绪队列。这样，当 A 线程跳出循环时，取出的就绪线程就是自己。这就要求线程的正文切换程序可以将一个线程切换到自己，Nachos 的线程正文切换程序 SWITCH 可以做到这一点，于是 A 线程实际上并没有被调下 CPU，而是继续运行下去了。

Nachos 线程管理系统的初步实现

1. 工具模块分析（文件list.cc list.h utility.cc utility.h）

工具模块定义了一些在 Nachos 设计中有关的工具函数，和整个系统的设计没有直接的联系，所以这里仅作一个简单的介绍。

List 类在 Nachos 中广泛使用，它定义了一个链表结构，有关 List 的数据结构和实现如下所示：

```
class ListElement {                                // 定义了 List 中的元素类型
public:
    ListElement(void *itemPtr, int sortKey);    // 初始化方法
    ListElement *next;                          // 指向下一个元素的指针
    int key;                                    // 对应于优先队列的键值
    void *item;                                // 实际有效的元素指针
};
```

其中，实际有效元素指针是(void *)类型的，说明元素可以是任何类型。

```
class List {
public:
    List();                                    // 初始化方法
    ~List();                                  // 析构方法
    void Prepend(void *item);                 // 将新元素增加在链首
    void Append(void *item);                  // 将新元素增加在链尾
    void *Remove();                           // 删除链首元素并返回该元素
    void Mapcar(VoidFunctionPtr func);        // 将函数 func 作用在链中每个元素上
    bool IsEmpty();                           // 判断链表是否为空
    void SortedInsert(void *item, int sortKey); // 将元素根据 key 值优先权插入到链中
    void *SortedRemove(int *keyPtr);          // 将 key 值最小的元素从链中删除，并返回该元素
};
```

```
private:
    ListElement *first;           // 链表中的第一个元素
    ListElement *last;           // 链表中的最后一个元素
};
```

其它的工具函数如 `min` 和 `max` 以及一些同调试有关的函数,这里就不再赘述。

2. 线程启动和调度模块分析（文件 `switch.s` `switch.h`）

线程启动和线程调度是线程管理的重点。在 Nachos 中,线程是最小的调度单位,在同一时间内,可以有几个线程处于就绪状态。Nachos 的线程切换借助于宿主机的正文切换,由于这部分内容与机器密切相关,而且直接同宿主机的寄存器进行交道,所以这部分是用汇编来实现的。由于 Nachos 可以运行在多种机器上,不同机器的寄存器数目和作用不一定相同,所以在 `switch.s` 中针对不同的机器进行了不同的处理。读者如果需要将 Nachos 移植到其它机器上,就需要修改这部分的内容。

2.1 ThreadRoot函数

Nachos 中,除了 `main` 线程外,所有其它线程都是从 `ThreadRoot` 入口运行的。它的语法是:

其中, `InitialPC` 指明新生成线程的入口函数地址, `InitialArg` 是该入口函数的参数; `StartupPC` 是在运行该线程是需要作的一些初始化工作,比如开中断;而 `WhenDonePC` 是当该线程运行结束时需要作的一些后续工作。在 Nachos 的源代码中,没有任何一个函数和方法显式地调用 `ThreadRoot` 函数, `ThreadRoot` 函数只有在线程切换时才被调用到。一个线程在其初始化的最后准备工作中调用 `StackAllocate` 方法(见本章 3.4),该方法设置了几个寄存器的值(`InterruptEnable` 函数指针, `ThreadFinish` 函数指针以及该线程需要运行函数的函数指针和运行函

数的参数)，该线程第一次被切换上处理机运行时调用的就是 `ThreadRoot` 函数。

其工作过程是：

1. 调用 `StartupPC` 函数；
2. 调用 `InitialPC` 函数；
3. 调用 `WhenDonePC` 函数；

这里我们可以看到，由 `ThreadRoot` 入口可以转而运行线程所需要运行的函数，从而达到生成线程的目的。

2.2 SWITCH函数

Nachos 中系统线程的切换是借助宿主机的正文切换。`SWITCH` 函数就是完成线程切换的功能。`SWITCH` 的语法是这样的：

1 2

其中 `t1` 是原运行线程指针，`t2` 是需要切换到的线程指针。线程切换的三部曲是：

1. 保存原运行线程的状态
2. 恢复新运行线程的状态
3. 在新运行线程的栈空间上运行新线程

3. 线程模块分析（文件 `thread.cc` `thread.h`）

`Thread` 类实现了操作系统的线程控制块，同操作系统课程中进程管理中的 `PCB` (`Process Control Block`) 有相似之处。

`Thread` 线程控制类较 `PCB` 为简单的多，它没有线程标识 (`pid`)、实际用户标识 (`uid`)等和线程操作不是非常有联系的部分，也没有将 `PCB` 分成 `proc` 结构和 `user` 结构。这是因为一个 Nachos 线程是在宿主机上运行的。无论是系统线程和

用户进程，Thread 线程控制类的实例都生成在宿主机而不是生成在虚拟机上。所以不存在实际操作系统中 proc 结构常驻内存，而 user 结构可以存放在盘交换区上的情况，将原有的两个结构合并是 Nachos 作的一种简化。

Nachos 对线程的另一个简化是每个线程栈段的大小是固定的，为 4096-5 个字 (word)，而且是不能动态扩展的。所以 Nachos 中的系统线程中不能使用很大的栈空间，比如：

10000

可能会不能正常执行，如果需要使用很大空间，可以在 Nachos 的运行堆中申请：

10000

如果系统线程需要使用的栈空间大于规定栈空间的大小，可以修改 StackSize 宏定义。

Thread 类的定义和实现如下所示：

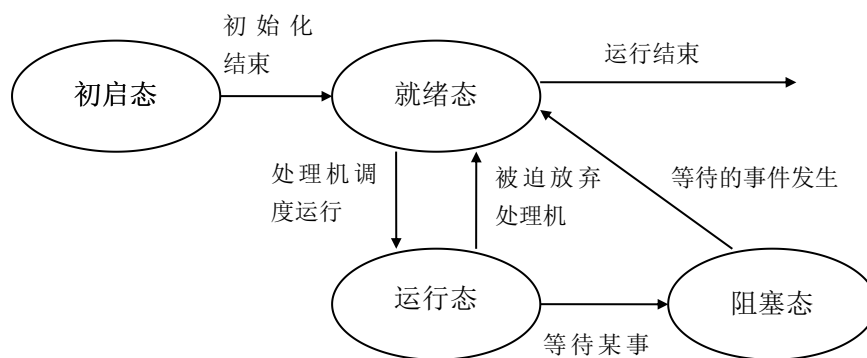
```
class Thread {
    private:
        int* stackTop;                // 当前堆栈指针
        int machineState[MachineStateSize]; // 宿主机的运行寄存器
    public:
        Thread(char* debugName);      // 初始化线程
        ~Thread();                    // 析构方法
        void Fork(VoidFunctionPtr func, int arg); // 生成一个新线程，执行
func(arg)
        void Yield();                // 切换到其它线程运行
        void Sleep();                // 线程进入睡眠状态
        void Finish();               // 线程结束时调用
        void CheckOverflow();        // 测试线程栈段是否溢出
        void setStatus(ThreadStatus st); // 设置线程状态
        char* getName() { return (name); } // 取得线程名（调试用）
        void Print() { printf("%s, ", name); } // 打印当前线程名（调试用）
    private:
        int* stack;                  // 线程的栈底指针
        ThreadStatus status;         // 当前线程状态
        char* name;                  // 线程名（调试用）
        void StackAllocate(VoidFunctionPtr func, int arg);
// 申请线程的栈空间
```

```

#ifdef USER_PROGRAM
    int userRegisters[NumTotalRegs];           // 虚拟机的寄存器组
    public:
    void SaveUserState();                       // 线程切换时保存虚拟机寄存器
    void RestoreUserState();                   // 线程切换时恢复虚拟机寄存器组
    AddrSpace *space;                          // 线程运行的用户程序
#endif
}

```

线程状态有四种，状态之间的转换如图 3 6 所示：



Nachos 线程的状态转换发生

用户进程在线程切换的时候，除了需要保存宿主机的状态外，必须还要保存虚拟机的寄存器状态。UserRegisters[]数组变量和 SaveUserState(), RestoreUserState()方法就是为了用户进程的切换设计的。

Fork 方法

语法： void Fork (VoidFunctionPtr func, int arg)

参数： func: 新线程运行的函数

arg: func 函数的参数

功能： 线程初始化之后将线程设置成可运行的。

1. 申请线程栈空间

实现：

2. 初始化该栈空间，使其满足 SWITCH 函数进行线程切换的条件
3. 将该线程放到就绪队列中。

返回： 空

StackAllocate 方法

语法： void StackAllocate (VoidFunctionPtr func, int arg)

参数： func: 新线程运行的函数

arg: func 函数的参数

功能： 为一个新线程申请栈空间，并设置好准备运行线程的条件。

```

::

// 申请线程的栈空间
- 4 // 设置栈首指针
--
// 线程准备好运行后进行线程切换，会切换到 函数。
函数
// 将会开中断，并调用 成为一个独立的调度单位。
实现： // 设置栈溢出标志
// 设置 指针，从
开始运行

// 以上是为 作好准备， 将分别调用 ，
// 和 。

返回： 空

```

4. 线程调度算法模块分析（文件scheduler.cc scheduler.h）

该模块的作用是进行线程的调度。在 Nachos 系统中，有一个线程就绪队列，其中是所有就绪线程。调度算法非常简单，就是取出第一个放在处理机运行即可。由于 Nachos 中线程没有优先级，所以线程就绪队列是没有优先级的。读者可以在这一点上进行加强，实现有优先级的线程调度。

4.1 Run方法

语法： void Run (Thread *nextThread)

参数： nextThread: 需要切换运行的线程

功能： 当前运行强制切换到 nextThread 就绪线程运行

1. 如果是用户线程，保存当前虚拟机的状态
2. 检查当前运行线程栈段是否溢出。（由于不是每时每刻都检查栈段是否溢出，所以这时候线程的运行可能已经出错）

实现： 3. 将 nextThread 的状态设置成运行态，并作为 currentThread 现运行线程（在调用 Run 方法之前，当前运行线程已经放入就绪队列中，变成就绪态）
（以上是运行在现有的线程栈空间上，以下是运行在 nextThread 的栈空间上）

4. 切换到 nextThread 线程运行

5. 释放 `threadToBeDestroyed` 线程需要栈空间（如果有的话）
6. 如果是用户线程，恢复当前虚拟机的状态

返回： 空

5. Nachos主控模块分析（文件`main.cc` `system.cc` `system.h`）

该模块是整个 Nachos 系统的入口，它分析了 Nachos 的命令行参数，根据不同的选项进行不同功能的初始化设置。选项的设置如下所示：

一般选项：

- : 显示特定的调试信息
- : 使得线程可以随机切换
- : 打印版权信息

和用户进程有关的选项：

- : 使用户进程进入单步调试模式
- : 执行一个用户程序
- : 测试终端输入输出

和文件系统有关的选项：

- : 格式化模拟磁盘
- : 将一个文件从宿主机拷贝到 模拟磁盘上
- : 将 磁盘上的文件显示出来
- : 将一个文件从 模拟磁盘上删除
- : 列出 模拟磁盘上的文件
- : 打印出 文件系统的内容
- : 测试 文件系统的效率

和网络有关的选项：

- : 设置网络的可靠度（在 0-1 之间的一个小数）

- : 设置自己的
- : 执行网络测试程序

6. 同步机制模块分析（文件synch.cc synch.h）

线程的同步和互斥是多个线程协同工作的基础。Nachos 提供了三种同步和互斥的手段：信号量、锁机制以及条件变量机制，提供三种同步互斥机制是为了用户使用方便。

在同步互斥机制的实现中，很多操作都是原子操作。Nachos 是运行在单一处理器上的操作系统，在单一处理器上，实现原子操作只要在操作之前关中断即可，操作结束后恢复原来中断状态。

6.1 信号量 (Semaphore)

Nachos 已经实现了 Semaphore，它的基本结构如下所示：

```
class Semaphore {  
    public:  
        void P();                // 信号量的 P 操作  
        void V();                // 信号量的 V 操作  
    private:  
        int value;               // 信号量值 ( >=0)  
        List *queue;             // 线程等待队列  
};
```

信号量的私有属性有信号量的值，它是一个阀门。线程等待队列中存放所有等待该信号量的线程。信号量有两个操作：P 操作和 V 操作，这两个操作都是原子操作。

6.1.1 P 操作

1. 当 value 等于 0 时，
 - 1.1. 将当前运行线程放入线程等待队列。
 - 1.2. 当前运行线程进入睡眠状态，并切换到其它线程运行。
2. 当 value 大于 0 时，value--。

6.1.2 V 操作

1. 如果线程等待队列中有等待该信号量的线程，取出其中一个将其设置成就绪态，准备运行。
2. `value++;`

6.2 锁机制

锁机制是线程进入临界区的工具。一个锁有两种状态，**BUSY** 和 **FREE**。当锁处于 **FREE** 态时，线程可以取得该锁后进入临界区，执行完临界区操作之后，释放锁；当锁处于 **BUSY** 态时，需要申请该锁的线程进入睡眠状态，等到锁为 **FREE** 态时，再取得该锁。

锁的基本结构如下所示：

```
class Lock {
public:
    Lock(char* debugName);           // 初始化方法
    ~Lock();                         // 析构方法
    char* getName() { return name; } // 取出锁名（调试用）
    void Acquire();                  // 获得锁方法
    void Release();                  // 释放锁方法
    bool isHeldByCurrentThread();    // 判断锁是否为现运行线程拥有
private:
    char* name;                      // 锁名（调试用）
};
```

在现有的 Nachos 中，没有给出锁机制的实现，锁的基本结构也只给出了部分内容，其它内容可以视实现决定。总体来说，锁有两个操作 **Acquire** 和 **Release**，它们都是原子操作。

6.3 条件变量

条件变量和信号量与锁机制不一样，它是没有值的。（实际上，锁机制是一个二值信号量，可以通过信号量来实现）当一个线程需要的某种条件没有得到满足

时，可以将自己作为一个等待条件变量的线程插入所有等待该条件变量的队列；只要条件一旦满足，该线程就会被唤醒继续运行。条件变量总是和锁机制一同使用，它的基本结构如下：

```
class Condition {  
    public:  
        Condition(char* debugName);           // 初始化方法  
        ~Condition();                         // 析构方法  
        char* getName() { return (name); }    // 取出条件变量名（调试用）  
        void Wait(Lock *conditionLock);       // 线程进入等待  
        void Signal(Lock *conditionLock);     // 唤醒一个等待该条件变量  
线程      void Broadcast(Lock *conditionLock); // 唤醒等待该条件变量的线  
程  
        private:  
            char* name;                       // 条件变量名（调试用）  
};
```

在现有的 Nachos 中，没有给出条件变量的实现，条件变量的基本结构也只给出了部分内容，其它内容可以视实现决定。总体来说，条件变量有三个操作 Wait、Signal 以及 BroadCast，所有的这些操作必须在当前线程获得一个锁的前提下，而且所有对一个条件变量进行的操作必须建立在同一个锁的前提下。

三、理解 Nachos 中支持用户进程的机制

Nachos 对内存、寄存器以及 CPU 的模拟

Nachos 机器模拟很重要的部分是内存和寄存器的模拟。Nachos 寄存器组模拟了全部 32 个 MIPS 机（R2/3000）的寄存器，同时加上有关 Nachos 系统调试用

的 8 个寄存器，以期让模拟更加真实化并易于调试，对于一些特殊的寄存器说明如下：

寄存器名	编号	描述
StackReg	29	用户程序的堆栈指针
RetAddrReg	31	存放过程调用的返回地址
HiReg	32	存放乘法结果的高32位
LoReg	33	存放乘法结果的低32位
PCReg	34	当前PC指针
NextPCReg	35	下一条执行语句的PC指针
PrevPCReg	36	上一条执行语句的PC指针（调试用）
LoadReg	37	需要延迟载入的寄存器编号
LoadValueReg	38	需要延迟载入的寄存器值
BadAddReg	39	当出错陷入（Exception）时用户程序的逻辑地址

Nachos 用宿主机的一块内存模拟自己的内存。为了简便起见，每个内存页的大小同磁盘扇区的大小相同，而整个内存的大小远远小于模拟磁盘的大小。由于 Nachos 是一个教学操作系统，在内存分配上和实际的操作系统是有区别的。事实上，Nachos 的内存只有当需要执行用户程序时用户程序的一个暂存地，而作为操作系统内部使用的数据结构不存放在 Nachos 的模拟内存中，而是申请 Nachos 所在宿主机的内存。所以 Nachos 的一些重要的数据结构如线程控制结构等的数目可以是无限的，不受 Nachos 模拟内存大小的限制。

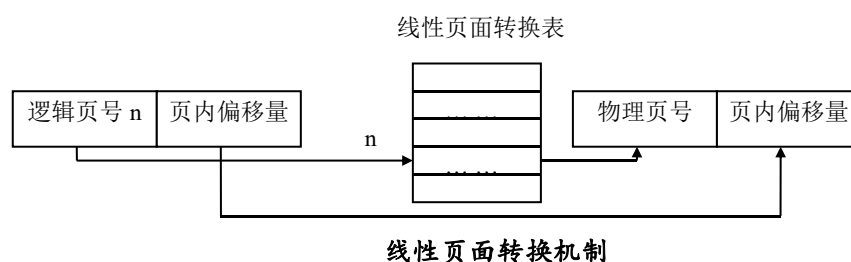
这里需要强调的是，此处 Nachos 模拟的寄存器组同 Thread 类（第三章第三节）中的 machineState[] 数组表示的寄存器组不同，后者代表的是宿主机的寄存器组，是实际存在的；而前者只是为了运行拥护程序模拟的。

在用户程序运行过程中，会有很多系统陷入核心的情况。系统陷入有两大类原因：进行系统调用陷入和系统出错陷入。系统调用陷入在用户程序进行系统调

用时发生。系统调用可以看作是软件指令，它们有效地弥补了机器硬件指令不足；系统出错陷入在系统发生错误时发生，比如用户程序使用了非法指令以及用户程序逻辑地址同实际的物理地址映射出错等情况。不同的出错陷入会有不同的处理，比如用户程序逻辑地址映射出错会引起页面的重新调入，而用户程序使用了非法指令则需要向用户报告等等。Nachos 处理的陷入有：

需要注意的是，虽然这里的很多方法和属性规定为 `public` 的，但是它们只能在系统核心内被调用。定义 `Machine` 类的目的是为了执行用户程序，如同许多其它系统一样，用户程序不直接使用内存的物理地址，而是使用自己的逻辑地址，在用户程序逻辑地址和实际物理地址之间，就需要一次转换，系统提供了两种转换方法的接口：**线性页面地址转换方法**和 **TLB 页面地址转换方法**。

线性页面地址转换是一种较为简单的方式，即用户程序的逻辑地址同实际物理地址之间的关系是线性的。在作转换时，给出逻辑地址，计算出其所在的逻辑页号和页中偏移量，通过查询转换表（实际上在使用线性页面地址转换时，`TranslationEntry` 结构中的 `virtualPage` 是多余的，线性页面转换表的下标就是逻辑



页号)，即可以得到实际物理页号和其页中偏移量。在模拟机上保存有线性页面转换表，它记录的是当前运行用户程序的页面转换关系；在用户进程空间中，也需要保存线性页面转换表，保存有自己运行用户程序的页面转换关系。当其被切换上模拟处理机上运行时，需要将进程的线性页面转换表覆盖模拟处理机的线性

页面转换表。线性页面转换的过程如图所示。

TLB 页面转换则不同，TLB 转换页表是硬件来实现的，表的大小一般较实际的用户程序所占的页面数要小，所以一般 TLB 表中只存放一部分逻辑页到物理页的转换关系。这样就可能出现逻辑地址转换失败的现象，会发生 `PageFaultException` 异常。在该异常处理程序中，就需要借助用户进程空间的线性页面转换表来计算出物理页，同时 TLB 表中增加一项。如果 TLB 表已满，就需要对 TLB 表项做 LRU 替换。使用 TLB 页面转换表处理起来逻辑较线性表为复杂，但是速度要快得多。由于 TLB 转换页表是硬件实现的，所以指向 TLB 转换页表的指针应该是只读的，所以 `Machine` 类一旦实例化，TLB 指针值不能改动。

在实际的系统中，线性页面地址转换和 TLB 页面地址转换只能二者取一，目前为简便起见，Nachos 选择了前者，读者可以自行完成 TLB 页面地址转换的实现。

一、用户程序空间（文件 `address.cc`, `address.h`）

Nachos 的用户进程由两部分组成：核心部分和用户程序部分。核心部分同一般的系统线程没有区别，它共用了 Nachos 的正文段和数据段，运行在宿主机上；而用户程序部分则有自己的正文段、数据段和栈段，它存储在 Nachos 的模拟内存中，运行在 Nachos 的模拟机上。在控制结构上，Nachos 的用户进程比系统线程多了以下内容：

```
int userRegisters[NumTotalRegs];    // 虚拟机的寄存器组
void SaveUserState();                // 线程切换时保存虚拟机寄存器组
void RestoreUserState();             // 线程切换时恢复虚拟机寄存器组
AddrSpace *space;                   // 线程运行的用户程序
```

其中，用户程序空间有 `AddrSpace` 类来描述：

```

:
//根据可执行文件构成用户程
序空间

//析构方法

// 初始化模拟机的寄存器组

// 保存当前机器页表状态

// 恢复机器页表状态

:

// 用户程序页表

// 用户程序的虚页数

```

在 Linux 系统中，使用 gcc 交叉编译技术将 C 程序编译成 R2/3000 可以执行的目标代码，通过 Nachos 提供的 **coff2noff** 工具将其转换成 Nachos 可以识别的可执行代码格式，拷贝到 Nachos 的文件系统中才能执行。

1.1 生成方法

```

语法:      AddrSpace (OpenFile *executable)
参数:      Executable:  需要执行代码的打开文件结构
功能:      初始化用户程序空间。
            1. 判断打开文件是否符合可执行代码的格式，如果不符合，出错返回
            2. 将用户程序的正文段、数据段以及栈段一起考虑，计算需要空间大小。如果
实现:      大于整个模拟的物理内存空间，出错返回。
            3. 生成用户程序线性页表。
            4. 将用户程序的正文段和数据段依次调入内存，栈段记录的是用户程序的运行
            状态，它的位置紧接于数据段之后。
返回:      空

```

目前 Nachos 在运行用户程序时，有如下的限制：

- 系统一次只能有运行一个用户程序，所以目前的线性转换页表比较简单，

虚拟页号同物理页号完全一样。当读者需要加强这部分内容时，需要增加内存分配算法。

- 系统能够运行的用户程序大小是有限制的，必须小于模拟的物理内存空间大小，否则出错。在虚拟内存实现以后，这部分内容也将做改动。

1.2 InitRegisters方法

语法: Void InitRegisters ()
参数: 无。
功能: 初始化寄存器，让用户程序处于可以运行状态。
实现: 设置 PC 指针、栈指针的初值，并将其它寄存器的值设置为 0。
返回: 空

1.3 SaveState方法

语法: Void SaveState ()
参数: 无。
功能: 存储用户程序空间的状态。
实现: 目前为空。
返回: 空

1.4 RestoreState方法

语法: Void RestoreState ()
参数: 无。
功能: 恢复处理机用户程序空间的状态。
实现: 赋值实现。
返回: 空

目前系统存储和恢复用户程序空间的实现非常简单，这是因为系统一次只能运行一个用户程序的局限和使用了线性页面转换表而决定的。

当用户程序空间初始化之后（设由 `space` 指针指向），真正的启动运行过程如下：

```
space -> InitRegisters();    // 初始化模拟机寄存器组
space -> RestoreState();     // 恢复处理机用户程序空间的状态，是将用户程
                             // 序空间的转换页表覆盖模拟机的转换页表
machine -> Run();            // 运行用户程序
```

二、系统调用（文件exception.cc, syscall.h, start.s）

系统陷入有两大原因：即系统调用和系统出错陷入。虽然计算机硬件本身提供了硬件指令，用户可以通过组织这些硬件指令来开发所有的系统软件和应用软件。但是在软件开发过程中我们发现，有些指令序列需要重复使用，正如在结构化程序开发中，函数可以被重复调用一样。我们将其中一些同硬件或者和操作系统功能有密切联系的一部分常用指令序列抽取出来，使它们成为操作系统本身的一部分，这就是所谓的系统调用。所以系统调用也被看作软件指令，它是在用户态下运行的程序和操作系统的界面。用户态程序可以通过系统调用获得操作系统提供的各种服务。

一般的 UNIX 操作系统提供几十条甚至上百条系统调用，系统调用的多少取决于操作系统的复杂程度及其主要的适用范围。只有用户态的程序才能进行系统调用，进行系统调用后的状态变成系统态，一般程序在系统态运行会有较高的优先级。系统调用作为操作系统的一部分，它们是长驻内存的。操作系统提供的系统调用太少，会影响用户程序的执行效率和用途；因为不是每种应用都会用到所有的系统调用，系统调用数目太多，则会使得操作系统的核心过于庞大而同样影响系统的效率。现代的很多操作系统提出了 mini-kernel 甚至 micro-kernel 的思想，在核心系统调用的选用方面比较谨慎，一般只选用最基本的部分，如网络、线程管理的与核心密切相关的部分，而将其它部分转化成开发系统的一部分，作为开发系统的标准函数提供，运行在用户态下。这样就可以减轻操作系统的负担，增加操作系统的灵活性、可配置性以及分布式应用开发的简便性等。

在 Nachos 中，系统调用用其它异常陷入的入口处理函数都是 ExceptionHandler 函数，只是陷入的类型为 SyscallException。

